

Intelligent Kubernetes Autoscaling Through Generative AI-Driven Workload Predictions

Nyoman Agus Nugraha Ginarsa
Department of Informatics
Institut Teknologi Sepuluh Nopember
 Surabaya, Indonesia
 6025232030@student.its.ac.id

Bagus Jati Santoso
Department of Informatics
Institut Teknologi Sepuluh Nopember
 Surabaya, Indonesia
 bagus@if.its.ac.id

Abstract— In the context of cloud-native architecture, the need for an efficient autoscaling mechanism is crucial to ensure service availability while avoiding resource waste. The Horizontal Pod Autoscaler (HPA) in Kubernetes has limitations in responding to real-time load spikes. This research aims to develop a prediction-based autoscaling approach using the Claude 3.7 Sonnet generative AI model via Amazon Bedrock. Historical CPU and memory data were collected by Prometheus at certain intervals, then converted to CSV format and sent to Claude 3.7 to generate a prediction of the number of pods needed in the next five minutes. The prediction results are then automatically applied to the Amazon EKS cluster via the Kubernetes API. The test results show that this approach can improve resource utilization efficiency and maintain service stability during traffic fluctuations, when compared to conventional HPA. This research also reveals that the accuracy of the prediction is highly dependent on the quality of the historical data, prompt, and AI model used. In the future, this research will expand the scope by adding other metrics such as latency, request rate, and I/O, and comparing the performance between generative AI models on Amazon Bedrock.

Keywords—autoscaling, Kubernetes, generative AI, Amazon Bedrock

I. INTRODUCTION

Cloud computing is defined as the management and provision of resources, software, applications, and information as services over the internet [1]. The application of cloud computing technology has experienced very rapid growth in recent years. This is marked by the increasing number of companies and government agencies migrating from on-premises infrastructure to cloud-based services. This shift is driven by the various advantages offered by cloud computing, such as elasticity in resource management, high service availability, and a pay-as-you-go payment model that allows users to only pay for the resources used.

Along with the adoption of cloud computing, container technology has also experienced a significant increase in popularity. Container is a lightweight abstraction of an operating system and the software application running for a particular service [2]. Containers are now the standard in the application deployment process because they are able to package all application needs, from source code, dependencies, libraries, and runtime in one consistent unit that can be run in various environments. One of the most widely used container orchestration systems today is Kubernetes. Kubernetes is the most popular container orchestration platform that enables users to create and run multiple containers in cloud environments [3]. Kubernetes facilitates the efficient management of container-based

applications, including on a production scale, by providing various features such as automatic scheduling, monitoring, and autoscaling mechanisms. Amazon Elastic Kubernetes Service (EKS) is an Amazon Web Services (AWS) integrated solution for managing Kubernetes clusters [4].

However, one of the main challenges in implementing Kubernetes is determining the optimal number of pods according to the application needs. Inaccuracy in determining the number of pods can result in two detrimental conditions such as over-provisioning and under-provisioning. Over-provisioning occurs when the amount of resources allocated exceeds the needs of the application, thereby increasing operational costs. Under-provisioning occurs when the resources provided are insufficient, which can lead to decreased service performance, instability, and even downtime.

Generative AI is an artificial intelligence system capable of generating images, text, and other media types with human prompts [5]. Generative AI technology has experienced rapid development and has now been applied in various fields, including data-based decision making. Generative AI has the ability not only to generate content but also to analyze data and provide predictions or recommendations based on historical patterns. This opens up new opportunities for overcoming the challenges of scaling services more adaptively and proactively.

Based on the literature study that has been conducted, AI and machine learning-based approaches have proven effective in improving autoscaling performance in the Kubernetes environment. However, most of these approaches still use traditional models that are less responsive to highly fluctuating workload dynamics. Therefore, this research proposes a new approach by adopting the generative AI technology in predicting the optimal number of pods on a Kubernetes cluster.

Claude is a new conversational AI system developed by AI safety startup Anthropic to be helpful, harmless, and honest [6]. Amazon Bedrock is a fully managed service from AWS that provides access to a variety of powerful foundation models (FMs) from leading AI companies like Anthropic through a single API. In this research, the Claude 3.7 Sonnet model from Anthropic accessed through the Amazon Bedrock service is used. This generative model is used to analyze historical data on application CPU and memory utilization collected through Prometheus, and then provides predictions regarding the number of pods that should be run in next five minutes. This prediction is expected to improve the efficiency of resource use while maintaining stable service availability.

This research is also an initial step in exploring the capabilities of generative AI in the context of cloud-native workload management. The developed system has been integrated with Amazon EKS, making it a solution that is not only automatic and adaptive but also ready to be used in a modern cloud ecosystem. With this approach, it is expected that the service scaling process can shift from traditional reactive methods to more proactive and prediction-based methods.

II. RELATED WORKS

In the context of optimizing autoscaling performance in Kubernetes clusters, several previous studies have addressed related topics. Research conducted by Shim et al. [7] developed a deep learning-based proactive autoscaling framework using the transformer model. This approach focuses on time-series workload prediction to determine the optimal number of pods. The goal is to reduce over-provisioning while maintaining application performance in dynamic load scenarios. Kuranage et al. [8] also proposed a deep-learning-based proactive autoscaling solution with a focus on Containerized Network Functions (CNF) in the Kubernetes environment. This model predicts CPU utilization and applies a new scaling algorithm to balance the quality of service (QoS) and operational cost efficiency. Pramesti & Kistijantoro [9] develop an autoscaler based on the prediction of the microservice application response time. This model dynamically calculates the number of pods needed based on the estimated response time, although its resource usage is higher than that of the conventional HPA.

Mishra et al. [10] used reinforcement learning (RL) to implement autoscaling in Kubernetes. The main goal of this approach is to overcome the limitations of the Horizontal Pod Autoscaler (HPA) in handling load spikes. The RL model is used to dynamically learn to scale parameters based on the current workload conditions. Santos et al. [11] also developed gym-hpa with reinforcement learning-based framework that takes into account the dependencies between microservices in Kubernetes. Using OpenAI Gym as a simulation environment, this framework can optimize resource usage and application response time.

Dogani & Khunjush [12] using federated learning (FedAgv-BiGRU) for proactive autoscaling in Kubernetes-based edge computing. The goal is to reduce latency and data transfer requirements by predicting workloads in a distributed manner without sacrificing data privacy. Toka et al. [13] proposed an AI-based autoscaling system that can adapt to the dynamics of application demand. Their approach uses a multi-prediction method to determine the optimal resource scale, which is very suitable for dealing with fluctuating workloads and increasing the level of compliance with SLAs. Tari et al. [14] presented a comprehensive review of autoscaling mechanisms in serverless computing environments. Although the focus of this research is on serverless, it discusses key concepts relevant to prediction-based resource management, including machine learning approaches.

Previous studies have shown that implementing AI and ML can improve autoscaling performance in Kubernetes clusters. Techniques such as deep learning-based workload prediction, federated learning, and adaptive scaling algorithms have proven effective in overcoming the shortcomings of traditional methods such as HPA, especially

in maintaining a balance between application performance, resource efficiency, and operational costs. This research offers a new approach using generative AI to generate more adaptive and contextual resource scaling predictions for real-time workload conditions. This research employs Claude 3.7 Sonnet generative AI to generate real-time, context-aware autoscaling decisions in Kubernetes, offering greater adaptability than conventional ML approaches.

III. PROPOSED METHOD

The methodology for implementing generative AI to predict optimal pod requirements in Kubernetes involves several stages, starting with the preparation of the Kubernetes cluster. This is followed by the data collection phase, the development of effective generative AI prompts, the execution of AI-based predictions for pod scaling, and finally, the validation of the results through testing. Figure 1 illustrates the overall workflow of the proposed method.

A. Kubernetes Cluster Preparation

This research uses Amazon Web Services (AWS) as a cloud infrastructure provider in implementing a system that uses generative AI to predict the optimal number of pods needed in a Kubernetes environment. Specifically, this research uses Amazon Elastic Kubernetes Service (Amazon EKS), a fully managed Kubernetes service from AWS that simplifies the process of deploying, managing, and scaling container-based applications. By using Amazon EKS, the complexity in provisioning and managing Kubernetes control components can be minimized, so that the main focus of the research can be directed at the AI-based autoscaling mechanism.

In the Kubernetes cluster, the sample application is run in a deployment object. This application is built with Go programming language with simple REST API. Resource allocation requests for this sample application per pod is 50m (milicore) for the CPU and 64Mi (mebibyte) for the memory. This application will be monitored periodically to obtain historical data on CPU and memory resource usage. The data becomes a knowledge base used by generative AI to predict future pod requirements. For monitoring purposes, this research uses Prometheus, an open-source monitoring and alerting toolkit that runs in a Kubernetes cluster. Prometheus is tasked with collecting historical data every 30 seconds that includes CPU usage, memory usage, CPU requests, memory requests, and the number of running pods. A diagram of the implementation of the generative AI to predict the optimal number of pods in Kubernetes is shown in Fig. 2.

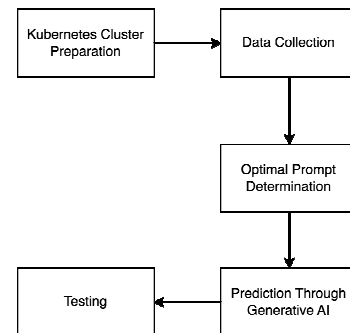


Fig. 1. Proposed Method of Implementation of The Generative AI to Predict Optimal Number of Pods in Kubernetes

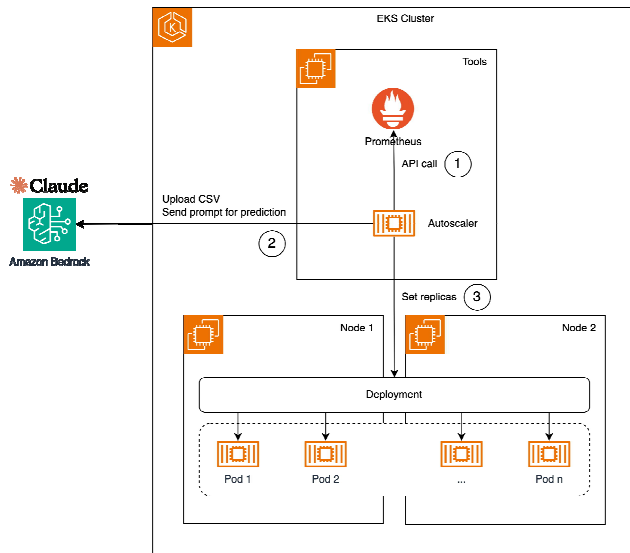


Fig. 2. Diagram of Implementation of The Generative AI to Predict Optimal Number of Pods in Kubernetes

B. Data Collection

The dataset that will be used as the knowledge base of generative AI comes from the history of CPU and memory utilization from sample applications every 30 seconds. Historical data is obtained from the Prometheus tools by accessing the Prometheus API and running the required queries with PromQL. Table I shows the queries used in the process of collecting CPU and memory utilization, as well as other information such as CPU requests, memory requests, and the number of replicas to produce more accurate predictions. The resulting query is the average value of the resource utilization used by all pods running in a deployment, so it uses a prefix in the selector.

After defining the queries needed in the data collection process, then continue with the creation of an autoscaler service to call the API from the Prometheus tools along with the query. The response from the value of each query is stored in a CSV file with the header timestamp, cpu_usage, memory_usage, cpu_requests, memory_requests, and replicas. Autoscaler service development is performed using the Go programming language and runs in a deployment. To ensure that the CSV file is saved, a persistent volume is attached to the container from the deployment service. Data retrieval is executed every 30 seconds using a cronjob.

TABLE I. PROMQL QUERY TO COLLECT RESOURCE UTILIZATION

Data	Query
CPU usage in millicores (m)	<code>avg(rate(container_cpu_usage_seconds_total{container!="", pod=~"rest-api"}[1m])) * 1000</code>
Memory usage in mebibyte (Mi)	<code>avg(container_memory_usage_bytes{container!="", pod=~"rest-api"}) / (1024 * 1024)</code>
CPU requests in milicore (m)	<code>avg(kube_pod_container_resource_requests{resource="cpu", pod=~"rest-api"}) * 1000</code>
Memory requests in mebibyte (Mi)	<code>avg(kube_pod_container_resource_requests{resource="memory", pod=~"rest-api"}) / (1024 * 1024)</code>
Number of pods running	<code>count(kube_pod_status_phase{phase="Running", pod=~"%s"})</code>

C. Optimal Prompt Determination

The prompt determination process is carried out so that the output produced by the generative AI is in accordance with expectations and produces accurate predictions. To obtain the appropriate prompt, prompt testing was carried out through the playground on the Amazon Bedrock console. The prompt used to predict the number of pods is shown in Table II.

In the prompt used to predict the number of pods, the addition of the sentence "Add if it feels lacking, reduce if it feels excessive" aims to ensure that pods can meet the resource needs of the application, reducing over-provisioning and under-provisioning. The addition of the sentence "No explanation needed, just the number of pods" emphasizes that generative AI does not provide reasoning as to why the AI model determines the number of pods, so that the output produced by generative AI is the recommended number of pods in the form of numbers, so that it can be used directly in adjusting the number of replicas in the sample application deployment. Prompt AI is also tested with dummy data that simulates several conditions, conditions when resource utilization is stable (no adjustment of the number of pods is required), resource utilization is close or more with the allocated resource (additional pod counts are required), and resource utilization is less than allocation (reduction of the number of pods is required). Dummy data testing was performed to simulate the real conditions that may occur in a production environment.

D. Prediction Through Generative AI

The pod requirement prediction process using the Claude 3.7 Sonnet model from Anthropic is performed through an autoscaler service integrated with Amazon Bedrock. This service is tasked with collecting metric data from the application (such as CPU usage, memory usage, CPU requests, memory requests, and replicas), packaging it in CSV format, and sending it along with an optimal prompt to Amazon Bedrock to get a prediction of the number of pods needed. Access to Amazon Bedrock is done by installing the AWS SDK on the autoscaler service and implementing IRSA (IAM Role for Service Account) so that the service can access Bedrock securely without storing credentials directly. After receiving a response from Bedrock in the form of the recommended number of pods, this service will update the number of replicas on the sample application deployment. To enable this, RBAC (Role-Based Access Control) configuration is required in Kubernetes, including the creation of a ServiceAccount, Role with permission to modify the deployment, and RoleBinding that associates the Role with the ServiceAccount. The autoscaler deployment is also configured to use this ServiceAccount so the replicas value in sample application deployment can be changed.

TABLE II. PROMPT TO PREDICT NUMBER OF PODS

[csv]
Based on the data in the following CSV file, how many pods need to be run for the next 5 minutes? Add if it feels lacking, reduce if it feels excessive
No explanation needed, just the number of pods

E. Testing

To ensure the system is functioning as intended, testing is performed using load simulation tools such as K6, which results in increased resource utilization, so it can be seen whether the system is able to dynamically adjust the number of pods according to the application needs. The system was tested under various conditions, including when CPU utilization increased significantly and when resource usage was very low.

The load testing scenario is structured in several stages to simulate realistic traffic patterns, namely starting with a medium load for a duration of 3 minutes and 50 virtual users, increasing to a high load for a duration of 3 minutes and 100 virtual users, reaching a peak load for a duration of 3 minutes and 150 virtual users, decreasing again for a duration of 3 minutes and 100 virtual users, and returning to the beginning for a duration of 3 minutes and 50 virtual users.

The endpoints used as targets in the load testing scenario are endpoints from a sample Go application deployment, designed to access and display query data from a MySQL database of 10.000 rows. The processing and sending of large amounts of data is intended to simulate heavy workloads on the application, especially on the backend processing and database access side.

IV. RESULTS AND DISCUSSION

In the initial stage, the process of collecting historical data on application resource utilization as a knowledge base for the generative AI model is carried out. This data was collected from Prometheus using a customized PromQL query to obtain information related to CPU usage, memory usage, CPU requests, memory requests, and the number of running pod replicas. Data are collected every 30 seconds and stored in CSV format by autoscaler service running in the Kubernetes cluster as deployment. The results of the data collection show that the system is able to record information consistently with the appropriate format, so that the data can be directly used in the prediction process. Table III shows the results of data collection in the CSV format.

After the data has been successfully collected, the process of predicting the number of pods is carried out by sending the content of CSV file along with an optimized prompt to the Claude 3.7 Sonnet model via Amazon Bedrock. The prompt is designed to request a prediction of the number of pods needed in the next five minutes without providing additional explanations, so that the prediction results can be directly used to adjust the replica value in the sample application deployment.

TABLE III. RESULT OF DATA COLLECTION

timestamp,cpu_usage,memory_usage,cpu_requests,memory_requests,replicas
1749350280,0.68,1.84,50,64,2
1749350310,0.71,1.85,50,64,2
1749350340,0.44,1.71,50,64,2
1749350370,0.65,1.92,50,64,2
1749350410,0.54,1.78,50,64,2
1749350440,0.54,2.16,50,64,2
1749350470,0.37,1.78,50,64,2
1749350500,0.41,1.84,50,64,2
1749350530,0.35,1.84,50,64,2
1749350560,0.41,1.85,50,64,2

The generative AI model provides a response in the form of a number representing the number of pods that should be run. The output is then used to automatically update the number of deployment replicas through an autoscaler service that has access to the Kubernetes API. To ensure the system working properly, Table IV shows the result of data collection when load testing scenarios using K6 is performed.

From Table IV, the test results showed that when the CPU utilization approached or over the CPU or memory requests, the AI model recommended adding pods automatically to avoid service disruption. However, when utilization was low, the system recommended reducing the number of pods to avoid wasting resources. This shows that the predictive approach successfully maintains a balance between service availability and resource efficiency.

TABLE IV. RESULT OF LOAD TESTING SCENARIOS

timestamp,cpu_usage,memory_usage,cpu_requests,memory_requests,replicas
1749353760,0.32,2.13,50,64,2
1749353790,0.37,2.14,50,64,2
1749353820,0.40,2.21,50,64,2
1749353850,0.39,1.98,50,64,2
1749353880,0.39,2.10,50,64,2
1749353910,0.42,1.83,50,64,2
1749353940,0.43,1.90,50,64,2
1749353970,0.42,1.65,50,64,2
1749354000,0.40,1.65,50,64,2
1749354030,0.42,1.78,50,64,2
1749354060,1.57,4.24,50,64,2
1749354090,2.27,6.52,50,64,2
1749354120,59.81,7.71,50,64,2
1749354150,55.90,8.59,50,64,3
1749354180,66.76,10.29,50,64,3
1749354210,49.02,11.87,50,64,3
1749354240,18.50,13.50,50,64,3
1749354270,19.14,12.73,50,64,3
1749354300,33.23,14.37,50,64,3
1749354330,47.05,15.67,50,64,3
1749354360,40.07,13.30,50,64,4
1749354390,41.57,14.74,50,64,4
1749354420,31.63,13.88,50,64,4
1749354450,39.10,12.22,50,64,4
1749354480,43.87,13.82,50,64,4
1749354510,43.03,11.66,50,64,4
1749354540,53.65,11.25,50,64,4
1749354570,56.74,11.63,50,64,4
1749354600,50.61,11.08,50,64,4
1749354630,44.05,8.44,50,64,5
1749354660,0.16,7.92,50,64,5
1749354690,0.25,6.19,50,64,4
1749354720,0.27,5.23,50,64,4
1749354750,0.26,5.23,50,64,3
1749354780,0.26,5.11,50,64,3
1749354810,0.29,4.79,50,64,3
1749354840,0.25,4.73,50,64,3
1749354870,0.28,3.72,50,64,3
1749354900,0.30,3.71,50,64,3
1749354930,0.27,4.04,50,64,2
1749354960,0.27,3.71,50,64,2
1749354990,0.20,3.44,50,64,2
1749355020,0.24,3.25,50,64,2
1749355050,0.28,3.22,50,64,2
1749355080,0.29,3.23,50,64,2
1749355110,0.26,3.22,50,64,2
1749355140,0.28,3.22,50,64,2
1749355170,0.27,3.69,50,64,2
1749355200,0.25,4.42,50,64,2
1749355230,0.28,4.42,50,64,2

The approach used in this research shows advantages over conventional autoscaling methods such as the Horizontal Pod Autoscaler (HPA). One of its main advantages is its ability to make proactive decisions based on historical data, rather than simply responding to current metrics. In addition, the good integration between Amazon EKS, Prometheus, and Amazon Bedrock allows the system to run automatically, adaptively, and fully managed. The test results prove that this approach can respond quickly to traffic spikes and maintain a stable service performance.

However, there are several limitations that need to be considered. Current prediction models only consider CPU and memory metrics without including other parameters such as latency, request rate, or disk I/O that also affect application performance. In addition, the prediction results are highly dependent on the quality and completeness of the available historical data. The prompts used also need to be carefully designed so that the model does not produce ambiguous or inconsistent output. Therefore, further testing and comparison with other generative models are needed to evaluate the stability and accuracy of the predictions under various conditions.

CONCLUSION AND FUTURE WORK

This research offers a new approach to optimizing the autoscaling performance on Kubernetes by utilizing generative AI technology. By utilizing the Claude 3.7 Sonnet model through the Amazon Bedrock service, generative AI is able to analyze historical resource utilization data (CPU and memory) collected through Prometheus and provide predictions for the number of pods that are more in line with the actual conditions of the application. This approach successfully reduces the risk of over-provisioning and under-provisioning, which have a direct impact on resource utilization efficiency and service stability. In addition, the full integration between Amazon EKS, Prometheus, and generative AI proves that cloud-native technology can be used synergistically to build an adaptive and autonomous system in managing application workloads. The test results show that this predictive system can quickly adjust the number of pods to changes in load, especially when there is a significant increase in traffic. With this approach, service scale management becomes more proactive compared to traditional reactive autoscaling methods.

Some things that can be further developed from this research include the addition of predictive parameters such as latency, request rate, and disk I/O to enrich the dataset and improve prediction accuracy. The evaluation of other generative AI models available on Amazon Bedrock services can also be done, one of which is DeepSeek R1 to determine the model with the best prediction performance. Applying the same approach to several application deployments with different workload characteristics to test the model generalization. Generative AI has great potential not only in resource optimization but also in driving full automation of workload management in cloud-native applications in the future.

REFERENCES

- [1] N. Tutubala and T. E. Mathonsi, "A Hybrid Framework to Improve Data Security in Cloud Computing," in *Big Data, Knowledge and Control Systems Engineering - Proceedings of the 7th International Conference, BdKCSE 2021*, Institute of Electrical and Electronics Engineers Inc., 2021. doi: 10.1109/BdKCSE53180.2021.9627294.
- [2] D. Cai, L. Qian, D. Xu, and Z. Huang, "A Case Study of Container Behavior Analysis with Cloud Resource Profile," in *Proceedings - International Symposium on Parallel Architectures, Algorithms and Programming, PAAP*, IEEE Computer Society, 2021, pp. 169–172. doi: 10.1109/PAAP54281.2021.9720472.
- [3] E. Kim, K. Lee, and C. Yoo, "On the Resource Management of Kubernetes," in *International Conference on Information Networking*, IEEE Computer Society, Jan. 2021, pp. 154–158. doi: 10.1109/ICOIN50884.2021.9333977.
- [4] S. R. Rizvi, A. Lubawy, J. Rattz, A. Cherry, B. Killough, and S. Gowda, "A Novel Architecture of Jupyterhub on Amazon Elastic Kubernetes Service for Open Data Cube Sandbox," in *International Geoscience and Remote Sensing Symposium (IGARSS)*, Institute of Electrical and Electronics Engineers Inc., Sep. 2020, pp. 3387–3390. doi: 10.1109/IGARSS39084.2020.9323748.
- [5] S. Sai, A. Gaur, R. Sai, V. Chamola, M. Guizani, and J. J. P. C. Rodrigues, "Generative AI for Transformative Healthcare: A Comprehensive Study of Emerging Models, Applications, Case Studies, and Limitations," *IEEE Access*, vol. 12, pp. 31078–31106, 2024, doi: 10.1109/ACCESS.2024.3367715.
- [6] Y. A. Mohamed, A. H. Mohamed, A. Kannan, M. Bashir, M. A. E. Adiel, and M. A. Elsadiq, "Navigating the Ethical Terrain of AI-Generated Text Tools: A Review," *IEEE Access*. Institute of Electrical and Electronics Engineers Inc., 2024. doi: 10.1109/ACCESS.2024.3521945.
- [7] S. Shim, A. Dhokariya, D. Doshi, S. Upadhye, V. Patwari, and J. Y. Park, "Predictive Auto-scaler for Kubernetes Cloud," in *SysCon 2023 - 17th Annual IEEE International Systems Conference, Proceedings*, Institute of Electrical and Electronics Engineers Inc., 2023. doi: 10.1109/SysCon53073.2023.10131106.
- [8] M. P. J. Kuranage, E. Hanser, L. Nuaymi, A. Bouabdallah, P. Bertin, and A. Al-Dulaimi, "AI-assisted proactive scaling solution for CNFs deployed in Kubernetes," in *2023 IEEE 12th International Conference on Cloud Networking, CloudNet 2023*, Institute of Electrical and Electronics Engineers Inc., 2023, pp. 265–273. doi: 10.1109/CloudNet59005.2023.10490067.
- [9] A. A. Pramesti and A. I. Kistijantoro, "Autoscaling Based on Response Time Prediction for Microservice Application in Kubernetes," in *2022 9th International Conference on Advanced Informatics: Concepts, Theory and Applications, ICAICTA 2022*, Institute of Electrical and Electronics Engineers Inc., 2022. doi: 10.1109/ICAICTA56449.2022.9932943.
- [10] P. Mishra, S. Hans, D. Saha, and P. Moogi, "Optimizing Cloud Workloads: Autoscaling with Reinforcement Learning," in *IEEE International Conference on Cloud Computing, CLOUD*, IEEE Computer Society, 2024, pp. 217–222. doi: 10.1109/CLOUD62652.2024.00033.
- [11] J. Santos, T. Wauters, B. Volckaert, and F. de Turck, "gym-hpa: Efficient Auto-Scaling via Reinforcement Learning for Complex Microservice-based Applications in Kubernetes," in *Proceedings of IEEE/IFIP Network Operations and Management Symposium 2023, NOMS 2023*, Institute of Electrical and Electronics Engineers Inc., 2023. doi: 10.1109/NOMS56928.2023.10154298.
- [12] J. Dogani and F. Khunjush, "Proactive auto-scaling technique for web applications in container-based edge computing using federated learning model," *Journal of Parallel and Distributed Computing*, vol. 187, May 2024, doi: 10.1016/j.jpdc.2024.104837.
- [13] L. Toka, G. Dobreff, B. Fodor, and B. Sonkoly, "Adaptive AI-based auto-scaling for Kubernetes," in *Proceedings - 20th IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing, CCGRID 2020*, Institute of Electrical and Electronics Engineers Inc., May 2020, pp. 599–608. doi: 10.1109/CCGrid49817.2020.00-33.
- [14] M. Tari, M. Ghobaei-Arani, J. Pouramini, and M. Ghorbani, "Auto-scaling mechanisms in serverless computing: A comprehensive review," *Computer Science Review*, vol. 53. Elsevier Ireland Ltd, Aug. 01, 2024. doi: 10.1016/j.cosrev.2024.100650.